

Modulo OO: Programação Orienta a Objetos

Exercício #1: Definição da classe Conta

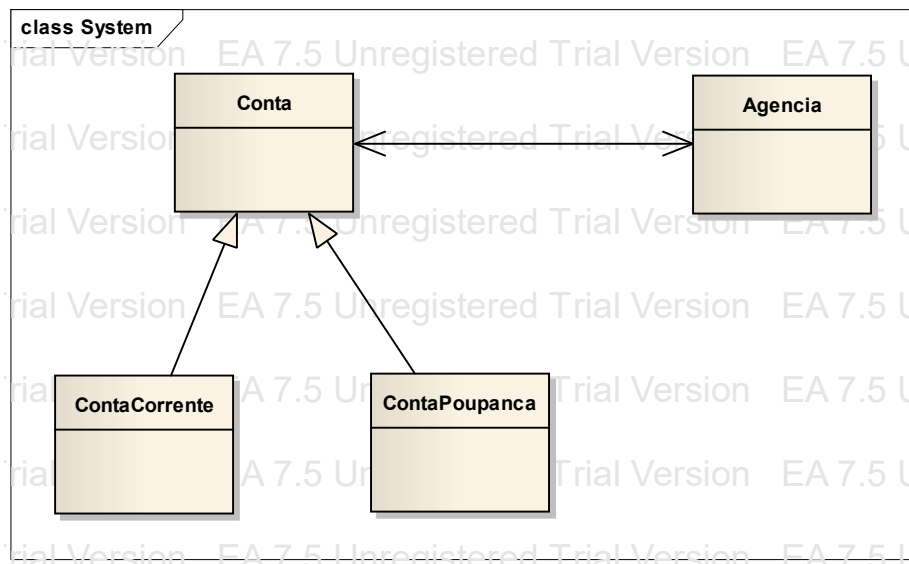
(Nível 2)

Objetivo

Nesse exercício será explorado a definição de classes contendo vários atributos e métodos. Posteriormente serão criados vários objetos do tipo Conta.

Orientações

Version 1: Diagrama de classes simplificado do projeto



Os atributos e métodos definidos em cada classe são:

Agência:

Atributos:

codigo	(inteiro)	Código da agência
descricao	(texto)	Descrição de uma agência
dataAbertura	(data)	Data de criação da agência

Métodos:

adicionar(Conta)	Acrescenta uma conta a agência
excluir(Conta)	Exclui uma conta da agência
listarContas()	Lista todas as Contas da agência

Conta:

Atributos:

numero (inteiro)	Número da conta
titular (String)	Titular da conta
saldo (real)	Saldo da conta
dataAbertura	Data de criação da conta

Métodos:

Gets e sets	Métodos acessores
debitar(double)	Faz débito em conta
creditar(double)	Faz crédito em conta
imprimeDadosConta()	Imprime todas informações da conta

Conta Poupança:

Atributos:

numero (inteiro)	Número da conta
titular (texto)	Titular da conta
saldo (real)	Saldo da conta
rendimento (real)	Rendimento de poupança

Métodos:

Gets e sets	Métodos acessores
debitar(double)	Faz débito em conta
creditar(double)	Faz crédito em conta
imprimeDadosConta()	Imprime todas informações da conta
imprimeRendimento()	Imprime o rendimento de poupança

Conta Corrente:

Atributos:

numero (inteiro)	Número da conta
titular (texto)	Titular da conta
saldo (real)	Saldo da conta
contaEspecial (boolean)	Indicador de conta especial
limiteCheque (double)	Limite de cheque especial

Métodos:

Gets e sets	Métodos acessores
debitar(double)	Faz débito em conta
creditar(double)	Faz crédito em conta
imprimeDadosConta()	Imprime todas informações da conta
imprimeRendimento()	Imprime o rendimento de poupança
getSaldoDisponivel()	Saldo total disponível (saldo +
limiteCheque)	
imprimeDadosConta()	Imprime todas informações da conta
imprimeLimite()	Imprime o limite de cheque especial

Exercícios etapa 1: Criação do ProjetoBanco e da classe Conta

1. Crie o projeto nomeado ProjetoBanco e implemente a classe Conta no projeto.
2. Inclua os métodos assessores e o método construtor para todos os atributos.
3. Inclua os métodos **toString** e **equals** na classe Conta. Duas contas serão consideradas iguais se tiverem o mesmo número de conta.
4. Inclua os métodos de instância.

No método Main da sua classe, inclua o código abaixo para criar vários objetos do tipo Conta. Observe a estrutura do método Main como exemplo abaixo:

```
public static void Main(string args[])
{
    Conta c1 = new Conta(1, "José", 500);
    c1.imprimeDadosConta();

    c1.debitar(12222);
    c1.imprimeDadosConta();

    Conta c2 = new Conta(2, "Marcos", 1500);
    c2.imprimeDadosConta();

    c2.debito(12222);
    c2.imprimeDadosConta();

    Conta c3 = new Conta(3, "Maria", 1500);
    c3.imprimeDadosConta();

    c3.debito(12222);
    c3.imprimeDadosConta();
}
```

O método saldo `imprimeDadosConta()` quando invocado para um objeto, a conta de FHC por exemplo, imprime a seguinte saída na tela:

```
-- Informações de Conta --
Número: 11
Titular: FHC
Saldo: 1000000.00 Saldo OK
```

Saída na tela quando o saldo está negativo

```
-- Informações de Conta --
Número: 13
Titular: Maria
saldo: -456.34 Atenção cliente seu saldo está negativo
```

Exercícios etapa 2:

Versão 2: Faça a checagem do valor do saldo para autorizar o debito

Para resolver o problema de fazer um débito sem saldo suficiente, introduza uma checagem. Se o valor a ser debitado é maior que o saldo, imprima uma mensagem "Saldo inválido!" na tela e não faça o débito. Caso o saldo seja suficiente o débito é realizado normalmente.